



universidad
de león

Robótica de Servicios

Práctica Final

Navegación Avanzada con Nav2, Voz y
Múltiples Planificadores

Máster en Robótica e Inteligencia Artificial

2025/2026

Iván Rodríguez Pérez

Descripción general

Se ha diseñado y desarrollado un sistema de navegación en ROS2 para la implementación de un sistema robótico capaz de navegar en un entorno de simulación en Gazebo. El sistema se ha implementado con el robot turtlebot3 en el entorno de simulación turtlebot world.

El sistema diseñado cumple con los requisitos de navegación permitiendo navegar a una secuencia de puntos definida en un archivo .yaml que contiene diferentes localizaciones dadas por coordenadas X e Y y orientación final del robot. La gestión de la navegación se realiza a través de la API de Nav2.

Además de realizar una navegación como la descrita, recibe y procesa comandos de voz para determinar las acciones que debe realizar. El robot permite recibir la siguiente lista de acciones:

- *continue*: para continuar navegando al siguiente punto.
- *go back*: para volver al punto anterior en la lista.
- *stop*: para detener la navegación
- *change planner*: para cambiar el planificador global
- *change controller*: para cambiar el local planner

En caso de que la orden de voz no haya sido entendida por el robot, utilizará por defecto la acción continue.

El desarrollo principal del proyecto se llevó a cabo en un contenedor de Docker en wsl2. Este enfoque generaba de base algunas limitaciones que no se hubieran encontrado si el equipo contase con Linux nativo.

Las funcionalidades que requerían de un periférico del portátil tales como micrófono o altavoces, fueron desarrolladas en el SO nativo del portátil para facilitar su acceso, desarrollando módulos comunicados a través de microservicios que imitan la arquitectura ROS2. Estos módulos por un lado realizaban la reproducción de un mensaje de texto en formato voz y por otro grababa y transcribía las órdenes del usuario para enviárselas de vuelta a la aplicación robótica.

El sistema de navegación se desarrolló utilizando Nav2. Se creó una instancia Navigator capaz de recibir instrucciones goToPose con las coordenadas del punto destino.

La aplicación completa esta compuesta por estos nodos:

Nodo	Host	Descripción
orchestrator_node	ROS2	Nodo que orquesta la ejecución de la aplicación robótica
displayer_node	ROS2	Nodo que sirve para mostrar información sobre la navegación a modo de dashboard
listener_node	ROS2	Nodo en ROS2 que hace de puente entre ROS2se comunica con el módulo de Windows encargado de capturar y transcribir los comandos de voz
speaker_node	ROS2	Nodo en ROS2 que se comunica con el módulo de Windows encargado de reproducir
turtlebot3	ROS2	Conjuntos de nodos y aplicaciones que permiten la ejecución de la simulación, incluyendo turtlebot3, Gazbo y Rviz
speaker_server	Win	Nodo encargado de convertir los mensajes generados por la aplicación robótica en voz. Tiene comunicación directa con los altavoces del equipo.
listener_server	Win	Nodo encargado de capturar y transcribir los mensajes de voz enviados por el usuario y hacerlos accesibles a la aplicación robótica.

Desafíos y Resolución

Desafío número 1: acceso a altavoces y micrófono

Problema: no era posible acceder/conectar la aplicación robótica en ROS2 directamente al micrófono y altavoces del equipo.

Solución: Desarrollar nodos en el SO nativo del equipo que pudieran comunicarse con la aplicación robótica.

Desafío número 2: limitaciones de memoria del equipo carencia de GPU

Problema: El diseño original de la aplicación robótica completo requería del despliegue y ejecución simultáneo de los siguientes módulos/aplicaciones: Gazebo, Rviz, aplicación de ROS2 con sus módulos, transcriptor de voz y agente de IA para el procesamiento de la voz y ejecución de acciones. Este despliegue colapsaba el equipo y a pesar de que el modelo elegido para la implementación del agente era un LLM de tamaño relativamente bajo (LLaMa 3.1B), en el momento de procesar comandos para elegir las acciones colapsaba la aplicación y forzaba al reinicio.

Solución: Dado que se observó que el sistema de transcripción ofrecía buen rendimiento se definió una lógica manual que elegía la acción a tomar de acuerdo al contenido del mensaje según si contenía alguna palabra específica. De esta forma, no era necesario lanzar el servidor del LLM, de forma que se utilizaban menos recursos del equipo y se eliminaba el riesgo de que saturase y se congelase la simulación.

Desafío número 3: uso de global planners

Problema: para probar el uso de diferentes global planners se siguió la documentación oficial y se intentó utilizar el planner_server Lattice. A pesar de que se proporcionó el lattice_filepath, cuando el robot cambiaba a este planner la aplicación se detenía.

Solución: Finalmente se utilizaron otros planner_server y se descartó Lattice. Posiblemente el problema fuese debido a una incompatibilidad de los parámetros de Lattice con los parámetros del robot.

Respuesta a las preguntas

1. ¿Cómo se implementó la lectura de waypoints desde el archivo y cómo se integró con Nav2 Simple Commander?

Los waypoints se cargaron en un fichero .yaml. En el se definieron 5 posiciones objetivo definidas por sus coordenadas 2D (x, y), y la orientación final del robot (w). Durante la ejecución de la simulación, un nodo orquestador se encarga de leer y procesar cada waypoint de forma que configura y envía instrucciones al robot para que se desplace hacia él.

Para el procesamiento de cada waypoint, dado que cada waypoint se procesa de forma individual, las órdenes enviadas al robot en cada iteración contienen un único punto compuesto por (x, y, w). Los valores de (x, y, w) son asignados como atributos de clase a la clase PoseStamped.pose.position. Además de estos, se asigna como atributo de clase "map" para pose_stamped.header.frame_id. Una vez compuesto el waypoint como PoseStamped, se pasa a la instancia del navegador y se pasa como argumento al método goToPose. Una vez recibido, Nav2 se encarga de controlar la navegación para que el robot alcance el punto objetivo.

2. ¿Qué paquetes o herramientas se utilizaron para la síntesis y reconocimiento de voz en ROS 2?

Para realizar la síntesis de voz fue necesario crear un módulo externo a ROS2. En el contexto en el que se ha realizado el proyecto, el sistema robótico principal corre en Docker, que a su vez se gestiona desde WSL2, y todo ello dentro de un laptop Windows. En estas condiciones, el sistema robótico principal no tenía acceso a los periféricos del portátil que permitían obtener o desplegar voz en tiempo real tales como micrófono o altavoces.

Para solventarlo se creó un servidor en Windows con dos endpoints que imita el comportamiento de dos nodos en ROS2. La misión de cada nodo consiste en capturar la voz a través del micrófono del portátil y reproducir el mensaje que se envíe desde el sistema robótico. De esta forma cada servicio cumpliría la función de un nodo de ROS2. El acceso a cada servicio se realiza a través de sus respectivos endpoints, que imitarían el comportamiento de un topic o servicio.

El módulo de síntesis de voz utiliza la librería pyttsx3 que permite un acceso directo a los altavoces del portátil, y permite, convertir de una manera sencilla y rápida una cadena de caracteres en la reproducción de un audio.

El módulo de reconocimiento de voz se realizó en dos etapas. La primera etapa consistió en grabar un audio del mensaje transmitido por el usuario. El audio se

grababa con la librería sounddevice, que permite realizar la grabación de forma directa y posibilita especificar diferentes parámetros de la grabación, tales como la duración del audio. El audio generado es posteriormente cargado por la segunda etapa del módulo de reconocimiento de voz. Este se encarga de convertir el sonido de la grabación en una transcripción en formato de cadena de texto. La librería utilizada para ello fue FasterWhisper, que permite, utilizando modelos de Deep Learning, realizar las transcripciones de forma precisa. La librería permite elegir entre diferentes modelos diferenciados por tamaño que a su vez permiten realizar traducciones que mejoran conforme el tamaño del modelo transcriptor se incrementa. La implementación utilizada usa el modelo de mayor tamaño, y si bien es cierto que las transcripciones son muy precisas, el tiempo que toma para realizarlas (~1 minuto) podría ser un cuello de botella en un entorno de producción real en el que se requiere la respuesta rápida.

3. Describe el proceso para configurar y cambiar entre múltiples planificadores locales en Nav2 antes o durante la ejecución. ¿Cómo se realizaría este proceso desde fichero de configuración?

Para cambiar y configurar el planificador/controlador antes de la simulación, se puede hacer editando el fichero “waffle.yaml” ubicado en la carpeta “parms” del paquete turtlebot3. Este archivo tiene una sección que contiene la definición del nodo planner server y otra para controller server. Para cada uno de ellos se especifican los plugins disponibles mediante el parámetro “planner_plugins” y “controller_plugins” respectivamente. Para utilizar un plugin (ya sea para planner como para controller) es necesario que tenga definida su sección de configuración en la que aparezcan especificados sus parámetros. Si se desea cambiar el planificador asociado, se puede modificar la lista de “planner_plugins” y si se desea cambiar el controlador se modificaría la lista “controller_plugins”. Para cada uno de los casos se especificaría el plugin que se desea establecer, y se debería asegurar que ese plugin tiene sus especificaciones bien definidas a continuación.

Durante la navegación, se puede realizar el cambio de planificador y/o controlador usando la API de Nav2. Primero, será necesario crear un servicio cliente “planner_server/set_parameters” con la interfaz “rcl_interfaces/srv/SetParameters” para el planificador y otro “controller_server/set_parameters” con la interfaz “rcl_interfaces/srv/SetParameters” para el controlador. Una vez instanciados, la API permite enviar peticiones a cada uno de los servicios. La petición debe contener el id definido en el yaml para cada uno de los casos (planner_id / controller_id). Este parámetro modificará el valor de “planner_plugins”/“controller_plugins” y activará el planificador / controlador si

está debidamente definido. Este proceso se puede realizar antes o durante la navegación entre waypoints.

En la implementación actual se han definido varios planificadores globales, incluyendo Navfn, una variante de Navfn basada en A*, y Smac2D. Inicialmente también se probó el planificador Smac Lattice, pero fue descartado debido a incompatibilidades con el modelo del robot utilizado. También se han definido algunos controladores como DWB (incluido por defecto), MPPI Controller (basado en MPC), y RPPC.

4. ¿Qué criterios se utilizaron para cambiar de planificador local? Si lo has cambiado cómo afectó esto al comportamiento del robot cuando navega de forma normal entre Waypoints. ¿Y si se encuentra un objeto en su ruta? ¿Y si se queda atascado entre varios objetos que le impiden pasar?

Los experimentos realizados con el robot utilizaron varios criterios para realizar el cambio tanto del planificador como del controlador durante las navegaciones llevadas a cabo.

El primer en fue utilizar un criterio de cambio aleatorio para validar el sistema de cambio de planificador y controlador, y asegurarse de que las llamadas a la API y la construcción del fichero de configuración se habían realizado correctamente. Seguidamente, se desarrolló un sistema de supervisión de timeout y de estancamiento del robot. Este sistema establece un temporizador para dar tiempo al robot a alcanzar el objetivo. Si durante el tiempo dado, el robot es capaz de alcanzar el objetivo, todo sigue normalmente. Pero si el temporizador llega a cero y se activa el timeout, se analizará la causa: si el robot se ha desplazado será timeout puro, mientras que, si no se ha movido de su punto inicial, se asumirá que no es capaz de generar una trayectoria o de seguir una trayectoria válida que conecte la localización y orientación actual del robot con las localizaciones y orientaciones objetivo.

En este caso, se marcará el estado “STATIC” y se activarán una serie de acciones para salir del atasco según la configuración del robot:

- Si se ha optado por cambiar el planificador, el robot girará 180 grados de forma manual e intentará recalcular el camino de nuevo desde una nueva orientación con un nuevo planificador.
- Si se ha configurado para modificar el controlador, ante una situación de atasco se establecerá el controlador MPPIController

Cuando el robot navega normalmente, no se observaron diferencias notables dado que, el camino espacio disponible para navegar solía ser relativamente

amplio considerando las dimensiones del robot y su maniobrabilidad. Podría incluso decirse que en estas condiciones de navegación normal con restricciones reducidas tiene más impacto el global planner que el controlador local.

Cuando el robot se queda atascado entre diferentes objetos, la ruta trazada para esquivarlos depende del controlador desplegado. Esto es debido a que cada controlador aplica estrategias distintas de seguimiento de trayectoria, evasión de obstáculos y generación de comandos de velocidad.

Instrucciones para la ejecución del proyecto

Este proyecto se ha creado para permitir la simulación de un robot usando Nav2 y voz y superar las limitaciones surgidas por el uso de un equipo sin GPU y con Windows nativo.

1. Abre Docker Desktop y espera a que la aplicación esté disponible y conectada con WSL2.
2. Abre el directorio del proyecto en WSL2.
3. Copia el ejecutable archivo scripts/windows_server.py en una ubicación que desees de tu sistema de archivos de Windows.
4. En Windows, abre un terminal y ejecútala como administrador. Ve a la ubicación donde has copiado el Windows_server.py y ejecuta

```
$ python3 ./windows_server.py
```

Espera a que se haya levantado el servidor y tenga sus servicios disponibles.

5. Lanza el container. Una vez que el container está corriendo, abre dos terminales. En ambas, accede al workspace de ros2 (ros2_ws)
6. Terminal 1

6.1 compila los paquetes que forman parte de la aplicación con

```
$ colcon build
```

6.2 A continuación, y en la misma terminal haz source de los paquetes compilados con

```
$ source install/setup.bash
```

6.3 Lanza la simulación de turtlebot3 con el comando

```
$ source install/setup.bash
```

6.4 Exporta el modelo del robot usado para la simulación como variable de entorno con

```
$ export TURTLEBOT3_MODEL="waffle"
```

6.5 Lanza la simulación de turtlebot3 con el comando

```
$ ros2 launch turtlebot3_simulation.launch.py
```

6.6 Espera a que se carguen Gazebo, Rviz y todos los elementos de la simulación

7. Terminal 2

7.1 Haz source de los paquetes compilados

```
$ source install/setup.bash
```

7.2 Lanza el commander launcher del paquete commnader_bringup para lanzar simultáneamente todos los nodos que componen el control de la simulación y navegación

```
$ ros2 launch commander_bringup commander.launch.xml
```

8. La simulación está lanzada. Interactúa con el robot y dale comandos de voz para que navegue, cambie el planificador, cambie el controlador o vuelva a posiciones anteriores

Anexo

Investiga los diferentes planificadores locales disponibles en Nav2, como DWB, TEB y RPP.

Nav2 permite utilizar distintos algoritmos de planificación y control para llevar a cabo tareas de navegación.

Planificadores

El Planner Server es el encargado de calcular la trayectoria global del robot. Para ello, utiliza una representación del entorno obtenida a partir de los datos de sus sensores. Los algoritmos de planificación realizan búsquedas sobre un espacio en forma de cuadrícula o expanden los posibles estados del robot, teniendo en cuenta la viabilidad del camino y las restricciones existentes.

Controladores

El Controller Server genera las acciones de control necesarias para que el robot complete una tarea de navegación en su entorno local. Esto incluye el seguimiento de la trayectoria generada por el Planner Server y la evitación de obstáculos en tiempo real.

Tanto el Planner Server como el Controller Server alojan uno o varios plugins. Dependiendo del entorno, escenario o tarea, se puede seleccionar el plugin más adecuado.

Algoritmos de navegación en Nav2

Nav2 implementa algoritmos de navegación mediante plugins basados en action servers, entre los que se incluyen:

- DWB (Dynamic Window Approach): Puede utilizarse en robots diferenciales u omnidireccionales, tanto circulares como no circulares.
- TEB (Time Elastic Band): Algoritmo basado en control predictivo por modelo (MPC) que optimiza la trayectoria del robot minimizando el tiempo de navegación, la distancia a los obstáculos y garantizando la factibilidad respecto a las restricciones cinemáticas del robot.
- RPP (Regulated Pure Pursuit Controller): Implementa una variante del algoritmo pure pursuit que incorpora heurísticas de regulación para gestionar la evitación de colisiones y las restricciones de velocidad.